

SotsiaalAI - One Page App Summary

What it is

SotsiaalAI is a Next.js web app that provides role-based AI support for social-sector use through personal and room chat. It combines AI responses, voice tooling, and document-grounded assistance with account and subscription controls.

Who it is for

- Primary personas are social work professionals and people seeking help (CLIENT role), with admin users for analytics and RAG content operations.

What it does

- Runs personal AI chat with streamed responses and persisted conversation history.
- Supports room chat with live message sync over SSE and polling fallback.
- Provides voice features: dictation (STT) and reply playback (TTS) with provider fallback chains.
- Supports document analysis in chat using file upload, chunking, and relevance-based context injection.
- Manages invite-based room membership with self-paid vs sponsored billing logic.
- Includes registration/login, profile/subscription flows, and admin analytics plus RAG admin routes.

How it works (repo-backed architecture)

- Frontend: Next.js App Router pages and React components, centered around chat orchestration hooks and route-level pages.
- Backend: Next.js API routes under app/api for chat, rooms, invites, profile, subscription, STT/TTS, and RAG proxying.
- AI services: /api/chat uses OpenAI; a separate rag-service (FastAPI + Chroma) is called over HTTP for retrieval/analysis.
- Data layer: Prisma models on PostgreSQL store users, subscriptions, conversations, room messages, invites, and RAG docs.
- Realtime flow: SSE streams for assistant output and room updates, with optional Redis fanout support for room events.
- Not found in repo: explicit production topology diagram or infrastructure map beyond local Docker/Postgres and PM2 start script.

How to run (minimal)

- Install dependencies: npm install
- Start local Postgres: docker compose up -d postgres
- Create/update local tables: npm run db:push:local
- Start app: npm run dev
- Open <http://localhost:3000> ; for full RAG features, start rag-service using docs/RAG_SETUP.md steps.

Evidence files: package.json, docs/LOCAL_DEV.md, docs/chat-page-system-map.md, docs/RAG_SETUP.md, docs/routes.txt, prisma/schema.prisma, app/api/*